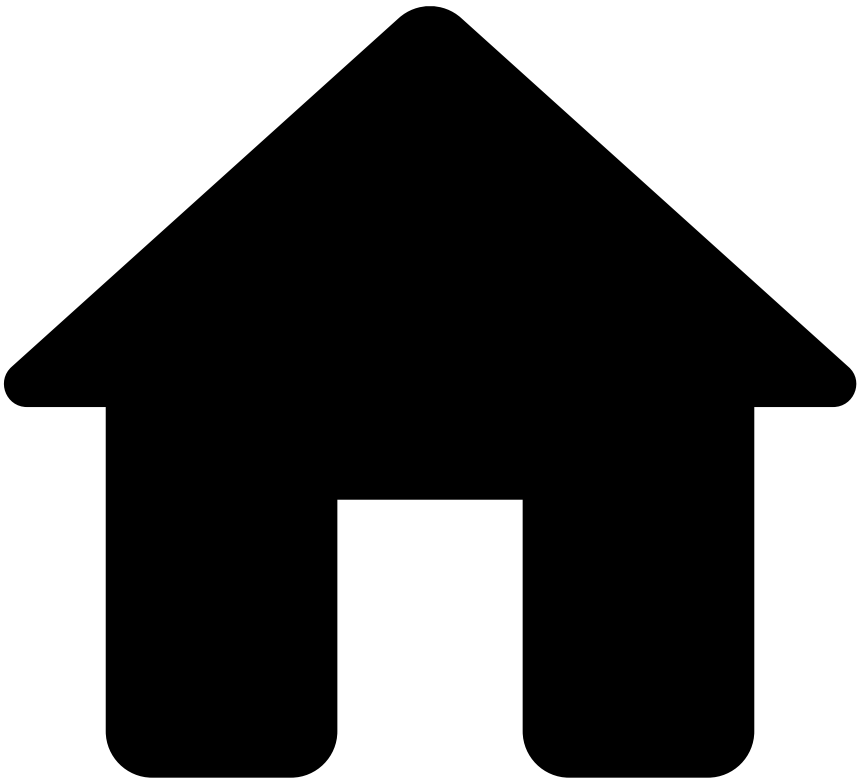


Rule Components

The following sections give you details about how a rule is evaluated when processing an event, and about the components that you can use when building a rule.

Variables and conditionâ□□

Rules contain the following parts that will be evaluated to decide if the current event matches it or not:

- **Variables**: the section that contains the preparation code.
- **Condition**: a boolean expression that determines if a rule matches the event.

Pulse uses rules, variables and conditions in the following way:

1. Pulse calculates the value of the variables for the event that is currenly processed.
2. Pulse evaluates the condition for the current event.
3. If the condition is true:
    - The actions defined by the rule are triggered.
    - An entry is added to score tracking.
    - The control flow defined in the rule dictates what to do next (check other rules, or stop processing, etc.).

If the condition is false, nothing happens and no entry is added to score tracking.

When you create rules, you are building complex expressions in Rule Definition Language (RDL). **RDL** is a Pulse domain-specific language that you can use to describe complex rules. The following sections give you more details about how variables and conditions are created in RDL.

Variablesâ□□

You can add variables to rules to avoid having large condition statements, which can be error prone and difficult to maintain.

The following example creates a variable holding the last 24 hours of transactions grouped by card number:

```
events24Hours = event[24 hours by cardnumber];
```

The following variable holds the sum of the amounts for the last 24 hours of transactions:

```
sumAmout24Hours = events24Hours.sum(amount);
```

Learn more about which RDL operators you can use in rules.

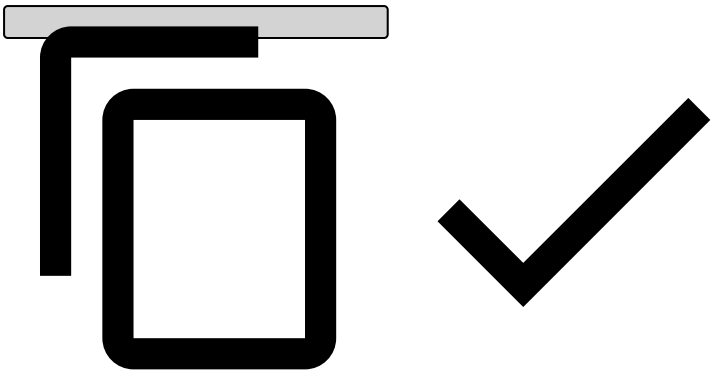
Note that the scope of variables is local to the rule that they are defined in, so a variable of a rule cannot be reused when designing other rules.

To declare global variables that are accessible to a set of rules, you can use environments when configuring a rules project.

Conditionâ□□

A **condition** is an expression that must return a boolean value, and that dictates whether the event triggers a rule. The condition expression must evaluate to *true* in order to match the event. The following examples show condition expressions:

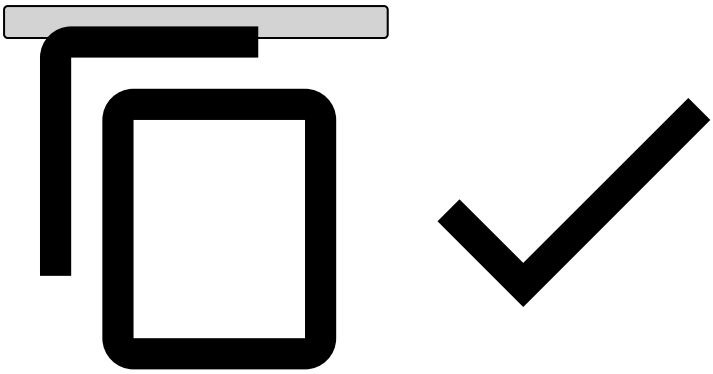
```
 //Match transactions where the sum of the amount for the current card in
the last 24 hours exceeds $20000**
//Assumes the existence of a sumAmout24Hours variable
sumAmout24Hours > 20000

//Match transactions with processing code 10 and amount less than $2**
event.processing_code == 10 and event.amount < 2

//Match transactions with processing code 10 and amount less than 2, or processing code 20 and
amount below $20
(event.processing_code == 10 and event.amount < 2) or (event.processing_code == 20 and event.amo
unt < 20)
```



The event variableâ

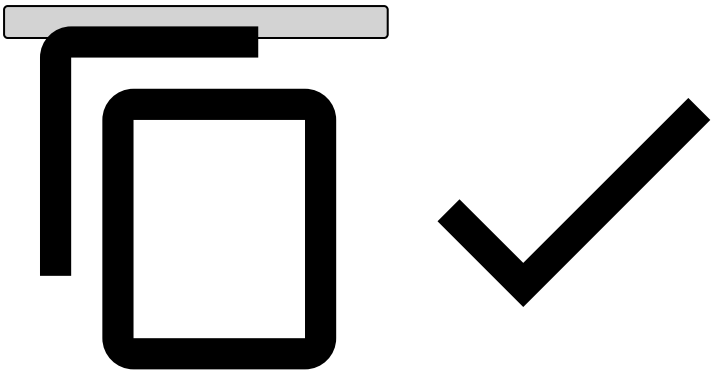
You can use the built-in *event* variable to access attributes of the current event. The following examples show how to do this:

```
 //Accessing fields of the current event
cardnum = event.cardnum;
amount = event.amount;
scoreIsAbove400 = event.score > 400;
```



By using the *event* variable, you can also access earlier events with a technique called **window**:

```
 //Getting the last five events by card number for the card number in the current event
lastEvents = event[5 tuples by cardnumber];

//by cardnumber creates a window with only the events that have card number
//equal to "event.cardnumber". Thus the last statement can be re-written as:
lastEvents2 = event[5 tuples by event.cardnumber];

//Last five events with terminal type X and Z
lastEventsWithTerminalTypeXZ = event[5 tuples where terminaltype == "X" or terminaltype ==
"Z" by cardnumber];

//Filter events by processing code
eventsWithProcessingCode5 = event.filter(e => e.processingcode == 5);
```

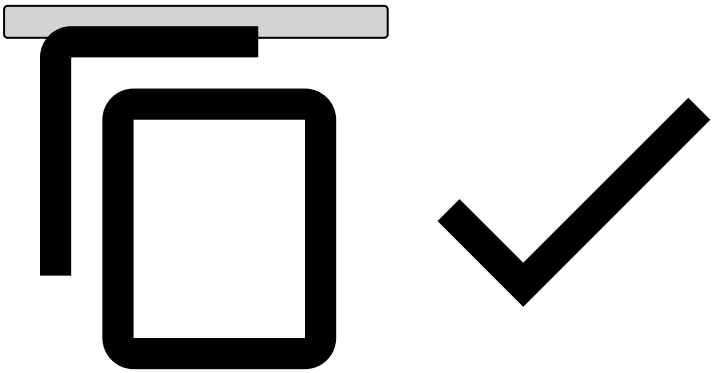


Learn more about the *event* variable and how you can use it to access historical data.

Integration with listsâ□□

You can use lists in rules. The following RDL code checks if an event attribute exists in a list:

```
 //blocklist is a previously defined list
isInBlockList = blocklist.contains(event.cardnumber);
```



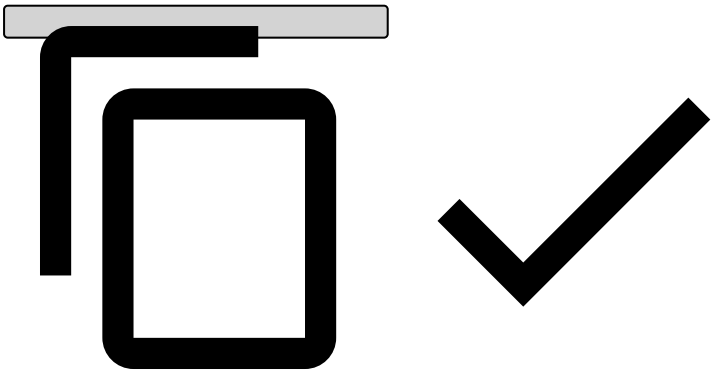
Learn more about how lists connect to rules.

Integration with enrichment functionsâ□□

Rules provide direct integration with data enrichment functions. This means that rule conditions can use the data enrichment service through user-defined functions or UDFs. Since the data enrichment service provides a large set of valuable data, that's often useful when deciding which path a transaction should follow in a live workflow.

The following is an example on how to evaluate if a given IP address belongs to the TOR network or not.

```
Enrichment.isTorNode(event.ipaddress)
```



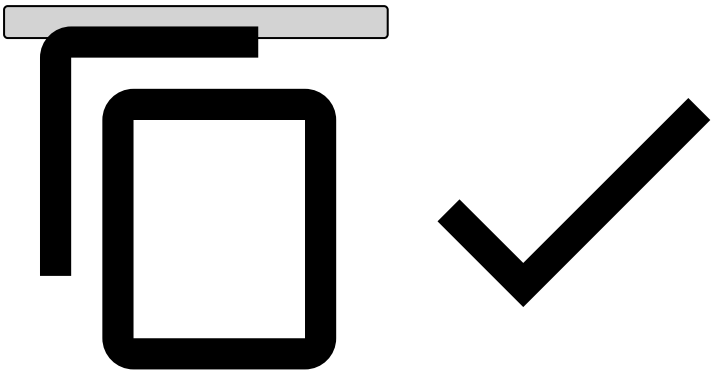
Integration with Feedzai TrustScoreâ

If Feedzai TrustScore is available in your environment, it can be used in rules to provide a risk score for the event being processed.

For example, the following RDL can be used to approve a transaction if the TrustScore-based fraud score for the current event is below the threshold:

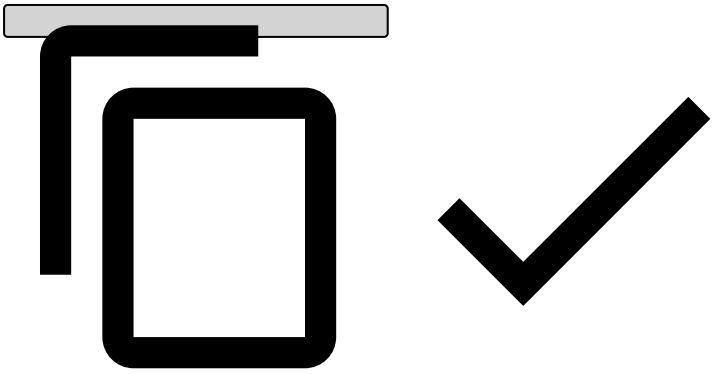
Variables:

```
 LOW_SCORE_THRESHOLD = Rules Thresholds["model_low_score"].threshold;
score = fraud_score * 1000;
```



Condition:

```
 (score ?? 0) < LOW_SCORE_THRESHOLD
```



Integration with environment metrics and variablesâ

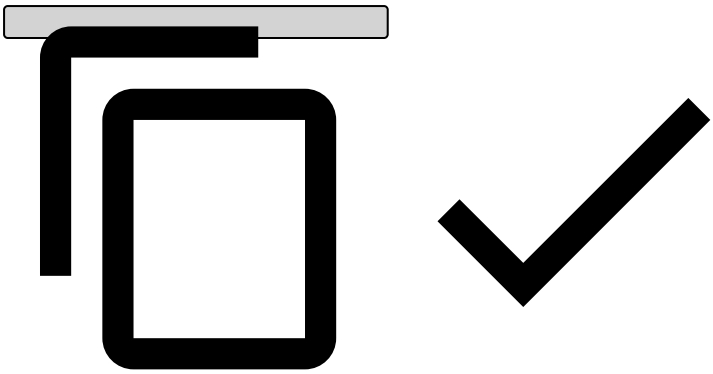
By default, a variable that you create in a rule can be used only in that specific rule. If you want to use the same variable in other rules as well, you must configure a so-called environment for that. An **environment** extends the scope of the variables present in it, it makes the variables available in every rule that belongs to that environment.

When creating a workflow, rules projects are represented by rule services. Each rule service has an isolated environment that you configure when adding the rule service to the workflow. The RDL variables (and also the PQL metrics) defined in that environment will be available in every rule of the environment.

Environment metricsâ

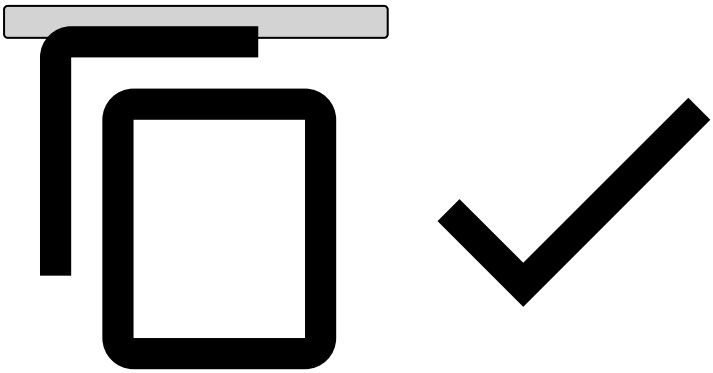
In the metrics code, an *input* stream is available. This stream represents the events of the respective workflow. The following example creates a metrics dictionary using PQL:

```
 // Sums the amount for each card number in the last 24 hours
sumAmount24Hours = from input[24 hours] group by cardnumber select amount:sum(amount);
```



The following example shows how to use this dictionary in RDL:

```
sumAmount24Hours[cardnumber].amount > 30000
```



RDL variablesâ□□

Creating RDL variables to be accessible in environments is the same as for rules themselves. The same variable will be available in every rule of the same rules project.

Templatesâ□□

Creating templatesâ□□

Sometimes it is necessary to define the same rule with different parameters several times. For example, you might want to generate alerts if the total transaction amount for a card number in the last 4 hours is above a certain threshold.

Pulse provides **template** mechanism that enables code reuse and helps you avoid writing the same logic several times.

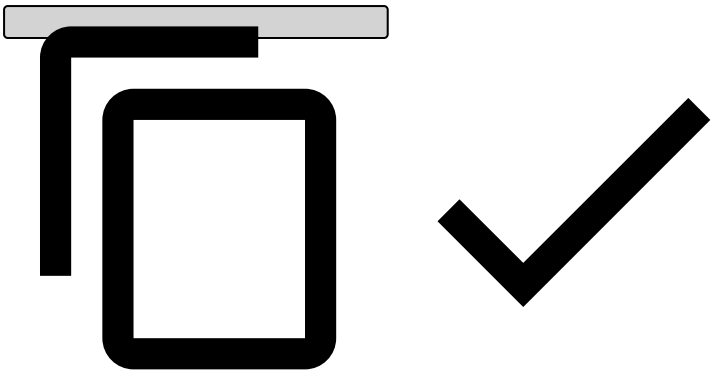
Templates allow a rule to be written without specifying all of its concrete parameters. Template parameters are used instead.

Each template parameter is characterized by:

- **Name** used to identify the parameters when applying the template to a rule
- **Type**
- **Default value**

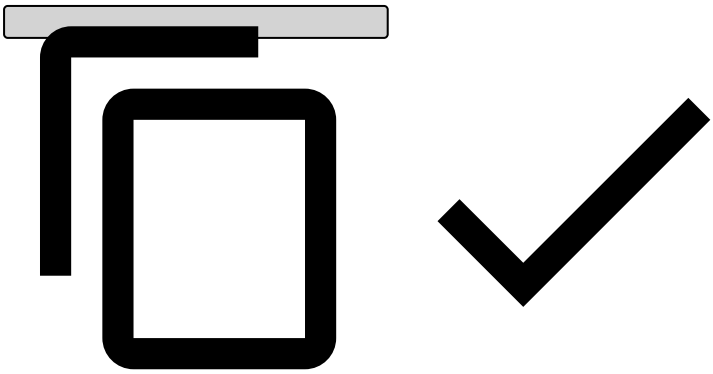
In terms of RDL code, templates work the same way as rules do, except that they allow defining template parameters. This is done using the `"$<TEMPLATE_NAME>"` notation:

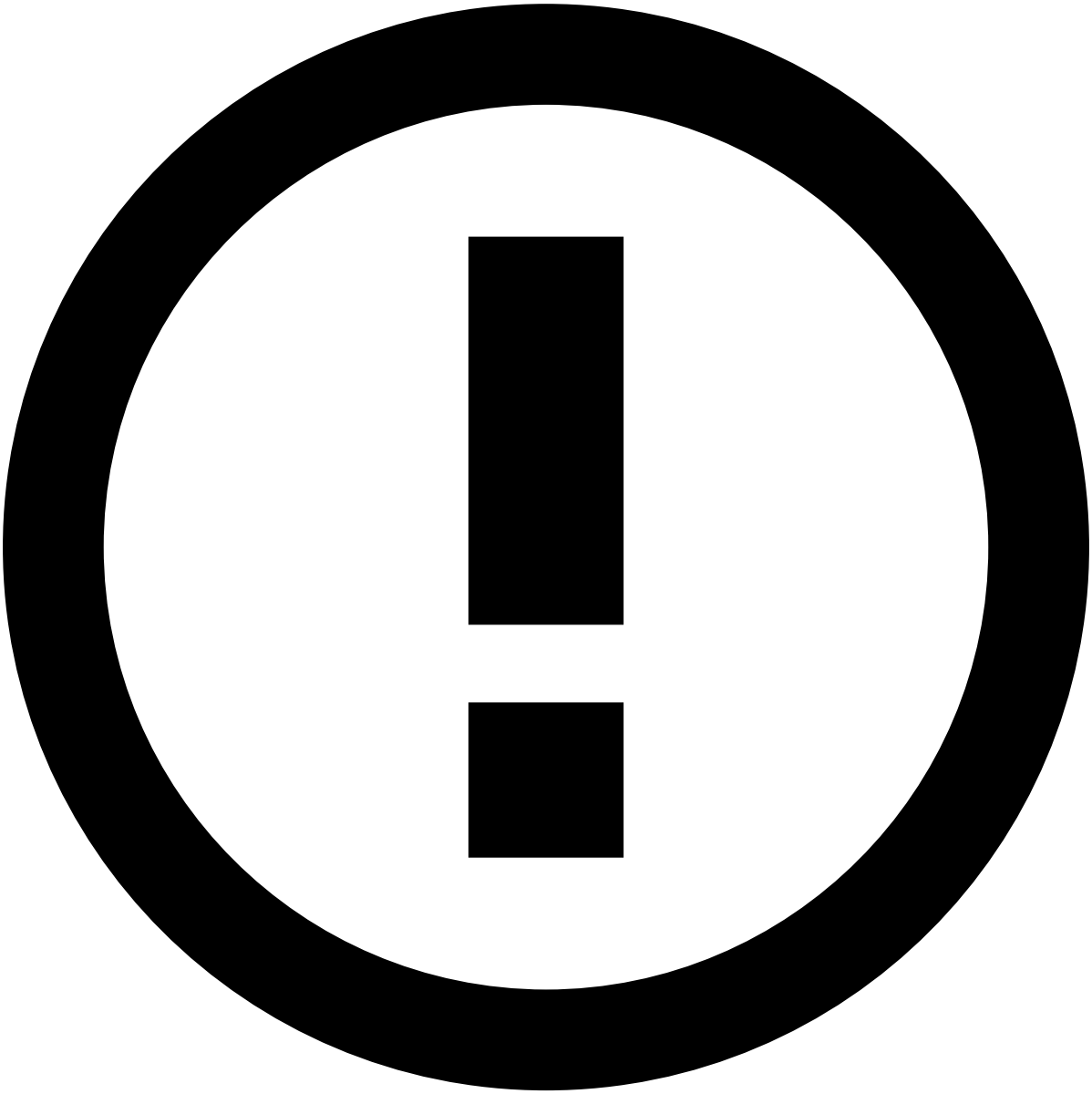
```
 //Parameterize the size of the window using template parameter nMin
lastEvents = event[$nMin minutes];
```



You can define template parameters both in the **Variables** and **Condition** fields. You can also define several template parameters as follows:

```
 //Parameterize window size with parameter nMin and condition with parameter cardnumber and
terminaltype
lastEvents = event[$nMin minutes where cardnumber == $cardnumber and terminaltype ==
$terminaltype];
```





info

Template parameters can only be used to replace literals.

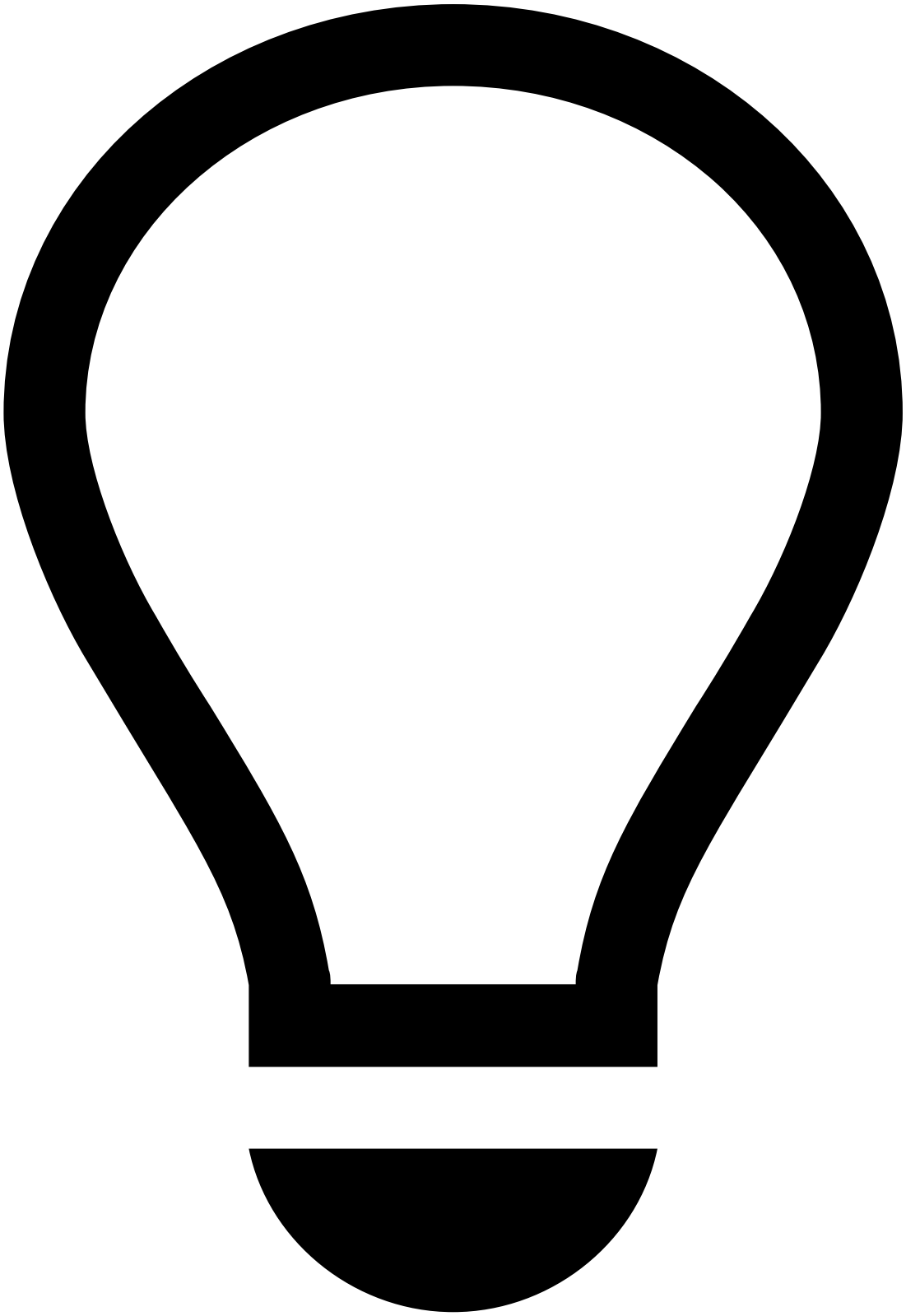
Rule explanation templatesâ□□

Rules explanations allow you to learn more about why a rule triggered as an event goes through a workflow, in a human-readable way.

When you create a rule inside a rules project you can define a template for the explanation as to why a rule triggered. Templates can be parameterized with available fields from the input data, lists and variables, similarly to when you are creating rules.

If Pulse renders the explanation when a rule is triggered, it will replace the added fields, lists and/or variables with their actual values.

The following image illustrates an example of a rule explanation template for an account opening use case.



tip

To generate explanations in a runtime workflow, activate explanations for each specific rule service in the workflow.